PROJECT TITLE: DISTRIBUTED MICROSERVICES BASED E-COMMERCE PLATFORM

1 . ABSTRACT

In many online shopping applications, product management, customer management, order processing, payment handling, and inventory management are handled within a single application or through tightly connected modules. As the number of users and transactions increases, maintaining the application, scaling individual components, and managing failures can become difficult. In this project, I propose to develop a Distributed Microservices Based E-Commerce Platform using Python, Django REST Framework, MySQL, and a web-based frontend.

The system provides an online shopping experience where customers can register, log in, browse products, search for items, add products to their shopping cart, place orders, make payments, and track their order status. The application follows a microservices architecture, in which major functionalities are separated into independent services such as User Management, Product Management, Cart Management, Order Management, Payment Management, and Inventory Management.

The platform provides two main role-based portals: Admin and Customer. The Admin manages product listings, categories, prices, inventory, and customer orders. Customers can manage their profiles, view product details, add items to their carts, place orders, and view their order history. An API Gateway acts as a common entry point for client requests and routes them to the appropriate microservices. The system also supports authentication, authorization, and communication between services through REST APIs.

This project aims to provide a scalable, maintainable, and reliable online shopping platform by separating business functionalities into independent services. It demonstrates how distributed application architecture can improve modularity, simplify maintenance, and support future expansion of an e-commerce business.

2 . OBJECTIVES

1. To develop a user-friendly online shopping platform for customers and administrators.
2. To manage customer registration, login, profiles, and secure authentication.
3. To store and manage product details, categories, prices, descriptions, and stock availability in a database.
4. To enable customers to search for products, view product information, and add products to their shopping carts.
5. To allow customers to place orders and view order history and order status.
6. To integrate a payment gateway for processing online payments and recording payment status.
7. To maintain accurate inventory records and update stock when orders are processed or cancelled.
8. To divide the application into independent microservices for user, product, cart, order, payment, and inventory management.
9. To implement an API Gateway that routes client requests to the appropriate microservices.
10. To provide an administrator dashboard for managing products, customers, orders, and inventory.
11. To improve application maintainability, scalability, and fault isolation through microservices architecture.

3 . EXISTING SYSTEM

Currently, many traditional e-commerce applications are developed using a monolithic architecture, where customer management, product management, cart operations, order processing, and payment handling are implemented within a single application. In some systems, these functions are closely connected and share the same application structure and database.

Limitations:

1. **Difficult Maintenance:** Changes in one module may require modifications, testing, and redeployment of the entire application.
2. **Scaling Limitations:** If product searches or order processing require more resources, scaling only that particular module can be difficult in a monolithic application.
3. **High Dependency:** Different modules may depend heavily on one another, making independent development and testing more complicated.
4. **Failure Impact:** A serious application-level failure can affect multiple e-commerce functionalities at the same time.
5. **Difficult Expansion:** Adding new features, such as advanced inventory tracking or a notification service, may require changes to several interconnected modules.
6. **Deployment Challenges:** Updating one functionality may require deploying the complete application instead of updating only the affected service.
7. **Limited Flexibility:** Different modules cannot easily be developed, deployed, or scaled independently when they are tightly integrated.

These limitations can make it harder to maintain and expand an e-commerce application as the number of customers, products, and transactions increases.

4 . PROPOSED SYSTEM

The proposed system is a web-based e-commerce platform developed using a microservices architecture. It separates major business functionalities into independent services that communicate through REST APIs.

How the System Works:

1. **User Registration and Authentication:** Customers create accounts and log in using their credentials. The system authenticates users and restricts access to authorized functionalities.
2. **Product Management:** The Product Service stores product names, descriptions, categories, prices, images, and availability information. Customers can browse and search for products.
3. **Shopping Cart Management:** Customers add products to their carts, update quantities, remove items, and view the total cost before placing an order.
4. **Order Processing:** When a customer confirms the purchase, the Order Service creates an order and maintains its status, such as Pending, Confirmed, Shipped, or Delivered.
5. **Payment Processing:** The Payment Service communicates with an external payment gateway to process payments and records successful, failed, or pending payment results. A simulated payment workflow can be used for an academic demonstration.
6. **Inventory Management:** The Inventory Service checks product availability and updates stock according to order-processing events. It helps prevent orders from exceeding available inventory.
7. **API Gateway:** The API Gateway provides a common entry point for client requests and routes them to the appropriate microservice. Authentication checks and other shared request-handling rules can also be applied here.
8. **Admin Dashboard:** The administrator can add, update, and remove products, manage categories, review orders, update inventory, and monitor basic sales information.
9. **Order Tracking:** Customers can view their order history and check the latest order status through their accounts.
10. **Independent Services:** Each microservice handles its own business responsibilities and can be maintained or deployed independently. If a service becomes temporarily unavailable, appropriate error handling helps prevent unnecessary failures in unrelated services.

Advantages of the Proposed System:

- Modular application architecture.
- Independent development and deployment of services.
- Easier maintenance and future feature expansion.
- Independent scaling of services according to demand.
- Better separation of business responsibilities.
- Improved authentication and access control.
- Organized product, order, payment, and inventory management.
- Better support for monitoring and troubleshooting individual services.

5 . ROLES AND RESPONSIBILITIES

1 . Admin :

- Logs in using authorized administrator credentials.
- Adds, updates, and removes product listings.
- Creates and manages product categories.
- Updates product prices, descriptions, and images.
- Monitors product stock and inventory information.
- Views customer orders and updates order status when required.
- Monitors customer accounts and authorized customer information.
- Views basic sales summaries and order reports.
- Manages the overall e-commerce platform.

2 . Customer :

- Registers for a new account and logs in securely.
- Views and updates personal profile information.
- Browses products and searches by name or category.
- Views product descriptions, prices, and availability.
- Adds products to the shopping cart and updates quantities.
- Places orders and completes the checkout process.
- Makes payments through the configured payment gateway or demonstration payment workflow.
- Views previous orders and tracks order status.
- Cancels eligible orders according to the system's rules.
- Logs out after completing shopping activities.

3 . System / Microservices :

- Authenticates users and verifies their access permissions.
- Routes incoming requests through the API Gateway.

- Processes product, cart, and order operations.
- Communicates between services through REST APIs.
- Checks inventory availability before confirming orders.
- Records payment results and updates order status.
- Handles errors and service communication failures.
- Maintains database records for the corresponding business functions.

6 . TECH STACK

Components	Technology
Backend Framework	Python,Django, Django REST Framework
Database	MySQL Database (with mysqlclient Connector)
Frontend UI	HTML, CSS, JavaScript, Bootstrap
API & Dev Tools	REST APIs,API Gateway,postman,Git,VS code